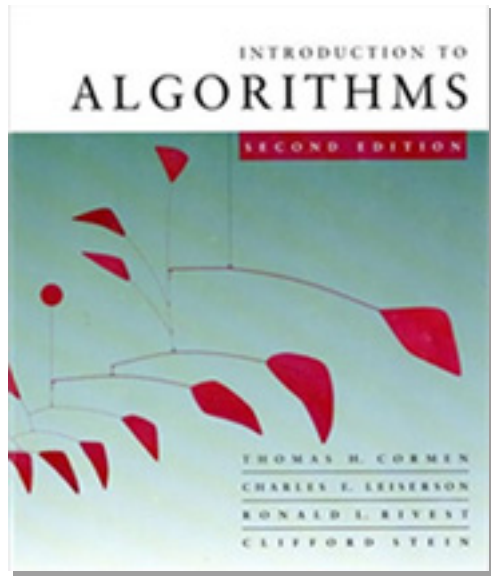


Introduction to Algorithms

6.046J/18.401J

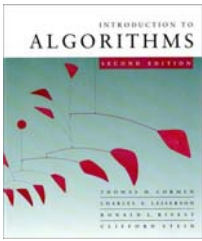


LECTURE 7

Hashing I

- Direct-access tables
- Resolving collisions by chaining
- Choosing hash functions
- Open addressing

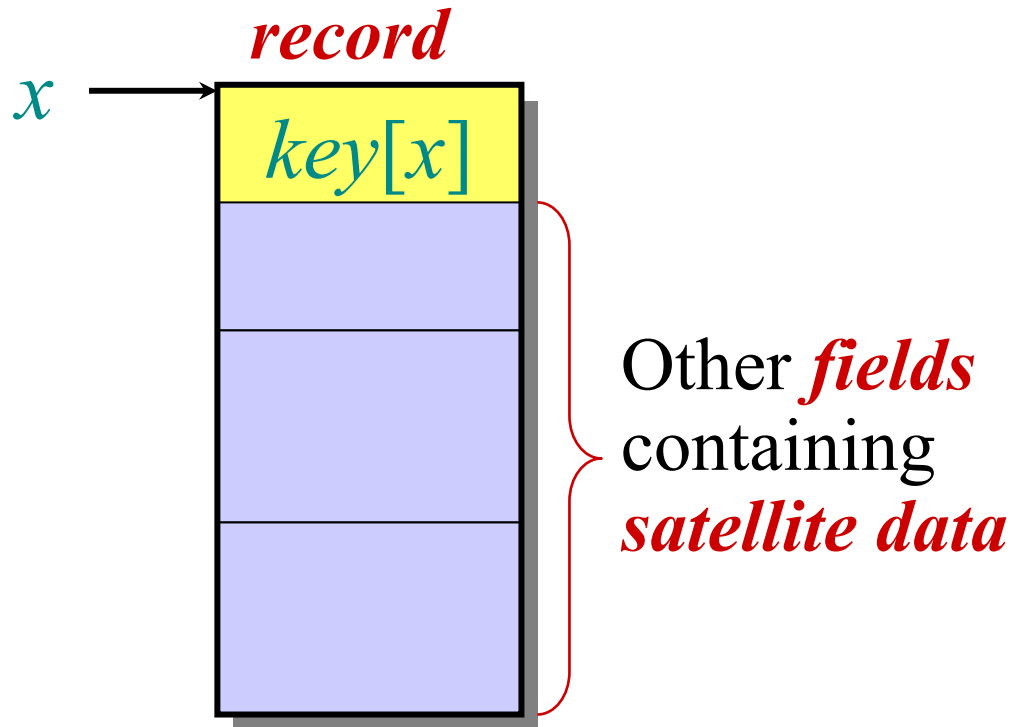
Prof. Charles E. Leiserson



Symbol-table problem

A Hash Table $\rightarrow$ a generalization of an array.
 $\hookrightarrow$ A dynamic set supporting dictionary operations

Symbol table S holding n **records**:

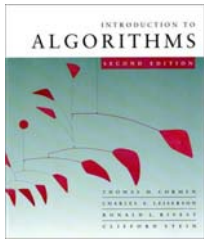


Operations on S :

- $INSERT(S, x) S \leftarrow S \cup \{x\}$
- $DELETE(S, x) S \leftarrow S - \{x\}$
- $SEARCH(S, k)$

$\hookrightarrow$ return x s.t.
 $key(x) = k$ or
 NIL if no such x

How should the data structure S be organized?



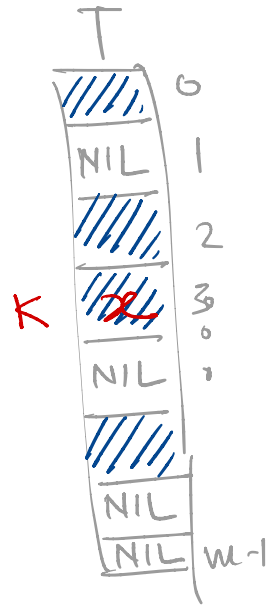
Direct-access table

$$K = \{x_0, x_2, x_3, \dots, x_{l-1}\}$$

$$\text{keys} = \{0, 2, 3, l, \dots\}$$

IDEA: Suppose that the keys are drawn from the set $U \subseteq \{0, 1, \dots, m-1\}$, and keys are distinct. Set up an array $T[0 \dots m-1]$:

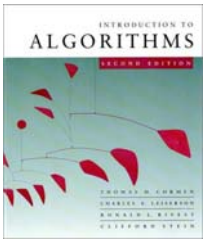
$$T[k] = \begin{cases} x & \text{if } x \in K \text{ and } \text{key}[x] = k, \\ \text{NIL} & \text{otherwise.} \end{cases}$$



Then, operations take $\Theta(1)$ time.

Problem: The range of keys can be large:

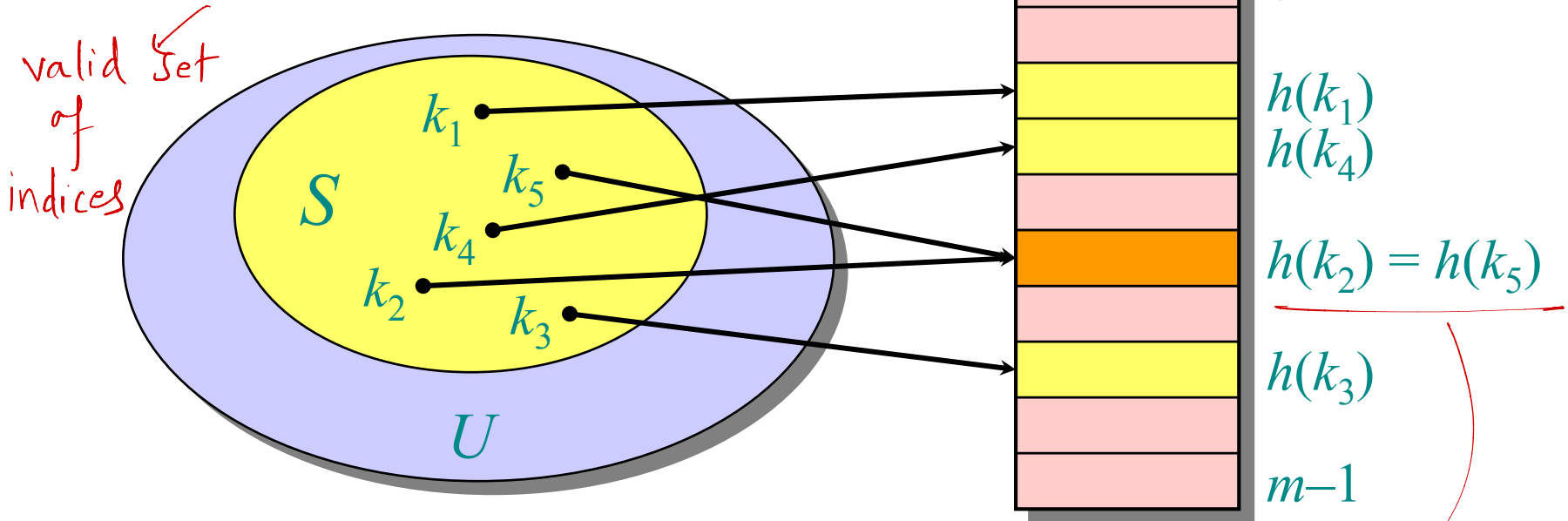
- 64-bit numbers (which represent 18,446,744,073,709,551,616 different keys),
- character strings (even larger!).



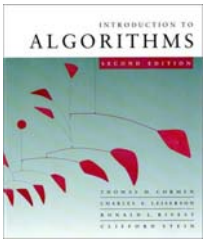
Hash functions

e.g. $h(k) = k \bmod m$

Solution: Use a *hash function* h to map the universe U of all keys into $\{0, 1, \dots, m-1\}$:

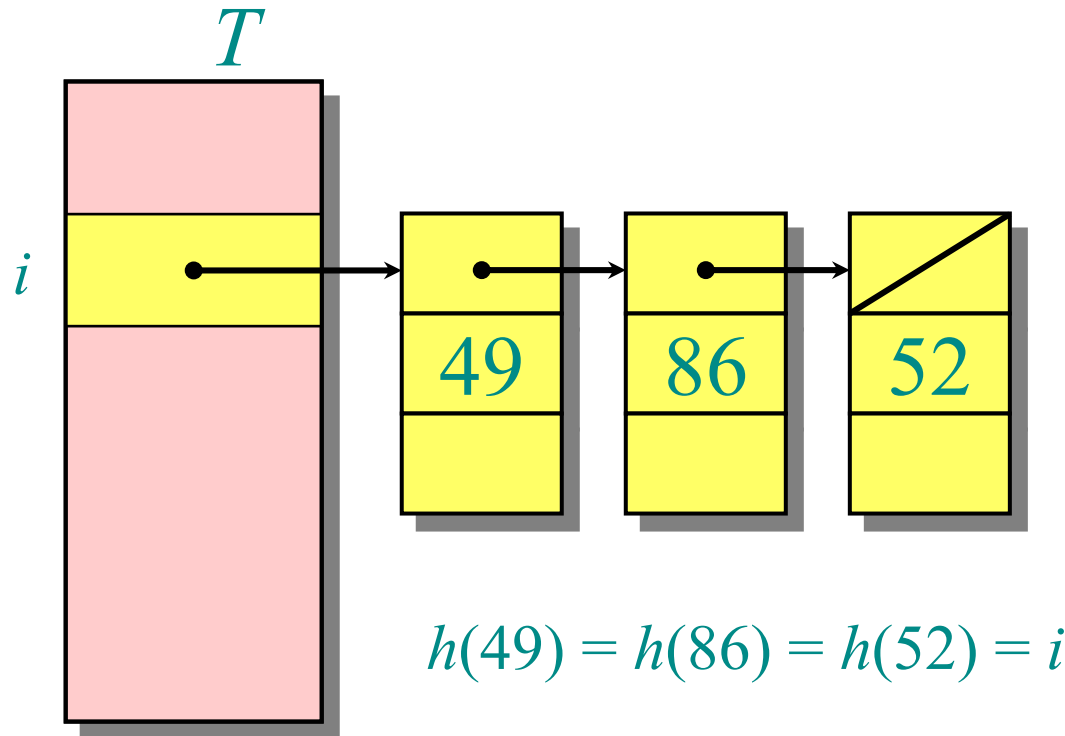


When a record to be inserted maps to an already occupied slot in T , a *collision* occurs.



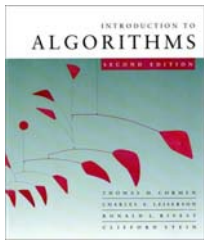
Resolving collisions by chaining

- Link records in the same slot into a list.



Worst case:

- Every key hashes to the same slot.
- Access time = $\Theta(n)$ if $|S| = n$



Average-case analysis of chaining

We make the assumption of *simple uniform hashing*:

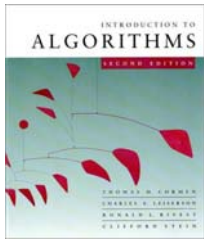
- Each key $k \in S$ is equally likely to be hashed to any slot of table T , independent of where other keys are hashed.

Let n be the number of keys in the table, and let m be the number of slots.

Define the *load factor* of T to be

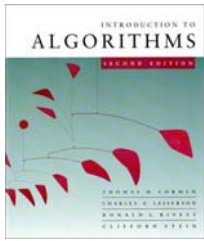
$$\alpha = n/m$$

= average number of keys per slot.



Search cost

The expected time for an *unsuccessful* search for a record with a given key is $= \Theta(1 + \alpha)$.



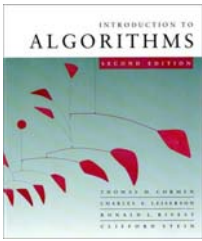
Search cost

The expected time for an *unsuccessful* search for a record with a given key is

$$= \Theta(1 + \alpha).$$

*search
the list*

*apply hash function
and access slot*



Search cost

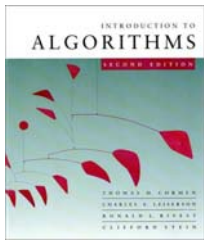
The expected time for an *unsuccessful* search for a record with a given key is

$= \Theta(1 + \alpha)$. *search the list*

apply hash function and access slot

Expected search time $= \Theta(1)$ if $\alpha = O(1)$,
or equivalently, if $n = O(m)$.

else $\Theta(\alpha)$



Search cost

The expected time for an *unsuccessful* search for a record with a given key is

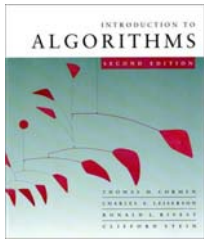
$= \Theta(1 + \alpha)$.

*search
the list*

*apply hash function
and access slot*

Expected search time $= \Theta(1)$ if $\alpha = O(1)$,
or equivalently, if $n = O(m)$.

A *successful* search has same asymptotic bound, but a rigorous argument is a little more complicated. (See textbook.)

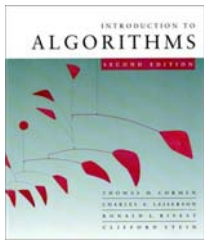


Choosing a hash function

The assumption of simple uniform hashing is hard to guarantee, but several common techniques tend to work well in practice as long as their deficiencies can be avoided.

Desirata:

- A good hash function should distribute the keys uniformly into the slots of the table.
- Regularity in the key distribution should not affect this uniformity.



Division method

Assume all keys are integers, and define

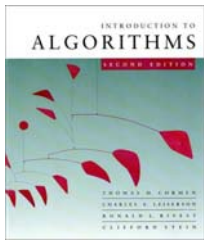
$$h(k) = k \bmod m. \equiv \text{rem}(k \div m)$$

Deficiency: Don't pick an m that has a small divisor d . A preponderance of keys that are congruent modulo d can adversely affect uniformity. *e.g. even keys with $d=2$ will leave all odd slots empty!*

factor $\Rightarrow$

Extreme deficiency: If $m = 2^r$, then the hash doesn't even depend on all the bits of k :

- If $k = 1011000111\underbrace{011010}_2$ and $r = 6$, then
 $h(k) = 011010_2.$ $h(k)$



Division method (continued)

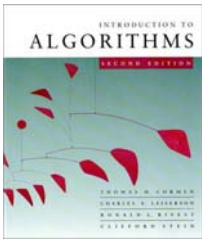
$$h(k) = k \bmod m.$$

Pick m to be a prime not too close to a power of 2 or 10 and not otherwise used prominently in the computing environment.

Annoyance:

- Sometimes, making the table size a prime is inconvenient.

But, this method is popular, although the next method we'll see is usually superior.



Multiplication method

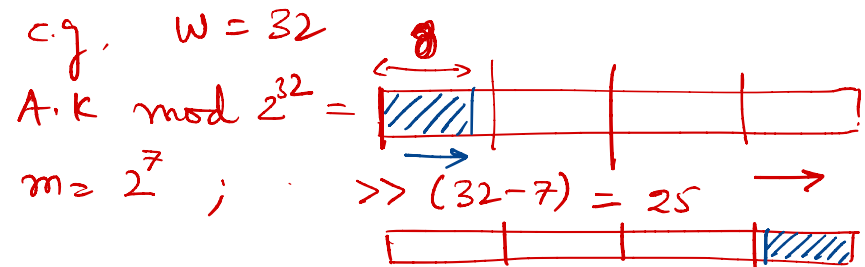
Assume that all keys are integers, $m = 2^r$, and our computer has w -bit words. Define

$$h(k) = (A \cdot k \bmod 2^w) \text{ rsh } (w - r),$$

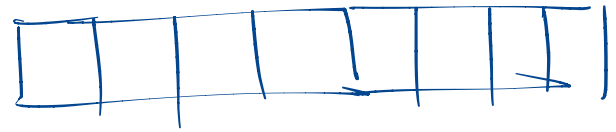
Handwritten notes: 'key' points to k ; 'multiplication will give a $2w$ wide number' points to $A \cdot k$; '>>' points to the rsh operator.

where **rsh** is the “bitwise right-shift” operator and A is an odd integer in the range $2^{w-1} < A < 2^w$.

- Don't pick A too close to 2^{w-1} or 2^w .
- Multiplication modulo 2^w is fast compared to division.
- The **rsh** operator is fast.



Let's say $w = 8$



$$A \in (2^7, 2^8)$$

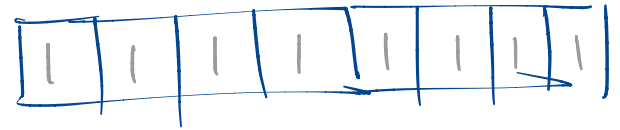


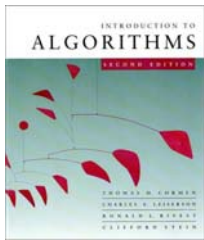
129



255

$$\in (128, 256)$$

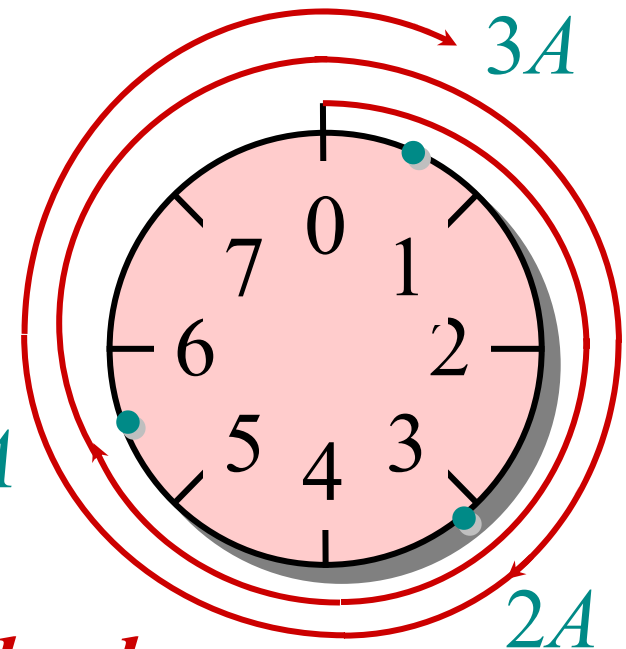
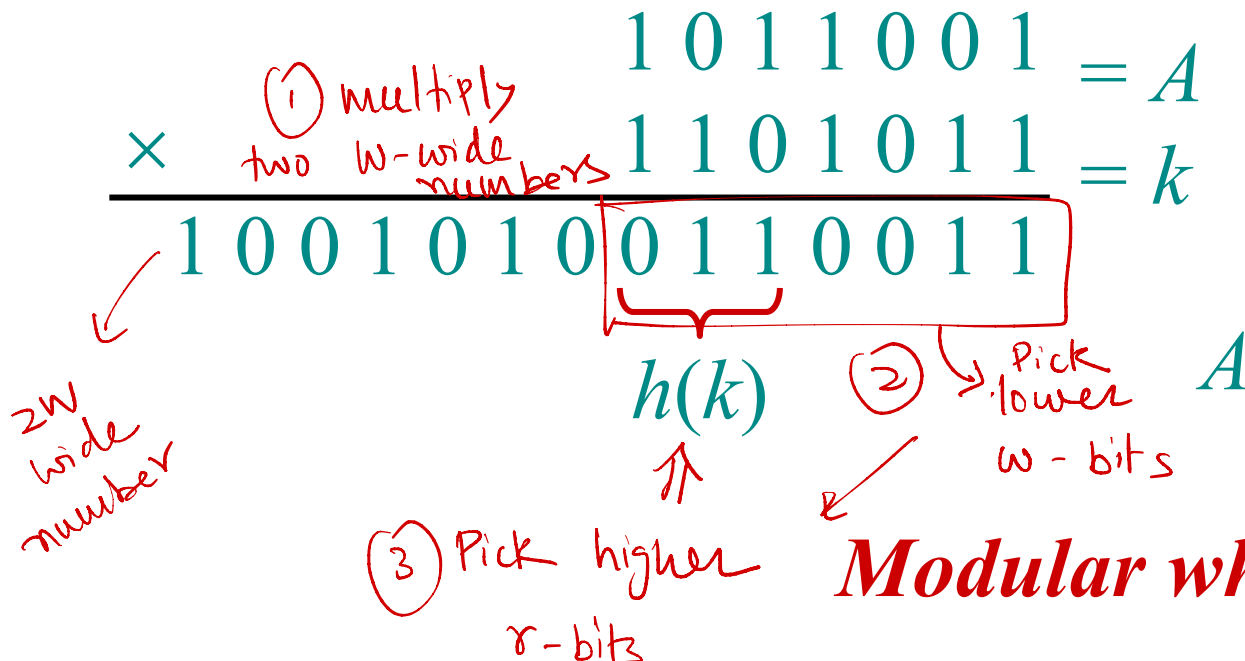




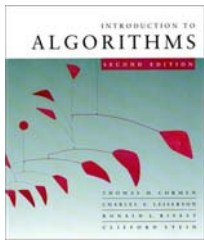
Multiplication method example

$$h(k) = (A \cdot k \bmod 2^w) \text{ rsh } (w - r)$$

Suppose that $m = 8 = 2^3$ and that our computer has $w = 7$ -bit words:



Modular wheel



Resolving collisions by open addressing

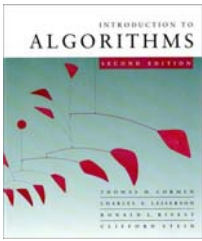
$\Rightarrow \# \text{ of keys} \leq \text{hash-table size}$

No storage is used outside of the hash table itself.

- Insertion systematically probes the table until an empty slot is found.
- The hash function depends on both the key and probe number:

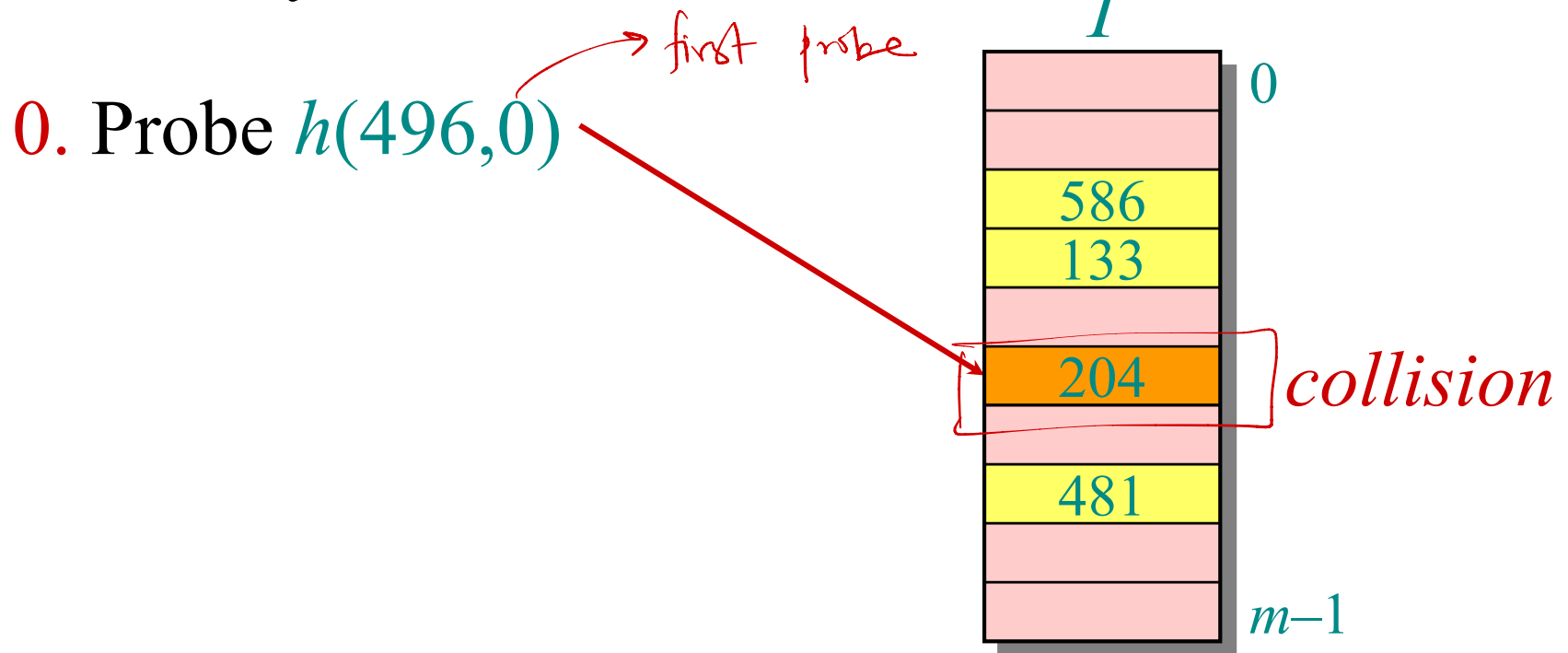
$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}.$$

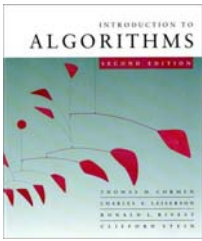
- The probe sequence $\langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$ should be a permutation of $\{0, 1, \dots, m-1\}$.
- The table may fill up, and deletion is difficult (but not impossible).



Example of open addressing

Insert key $k = 496$:





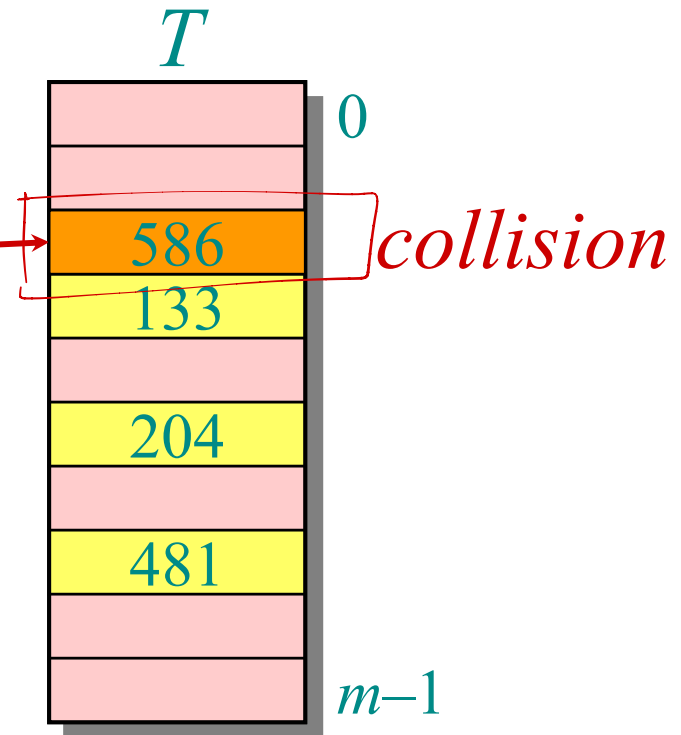
Example of open addressing

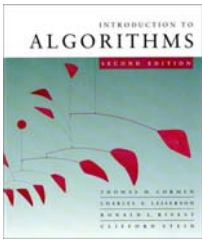
Insert key $k = 496$:

0. Probe $h(496, 0)$

1. Probe $h(496, 1)$

2nd probe





Example of open addressing

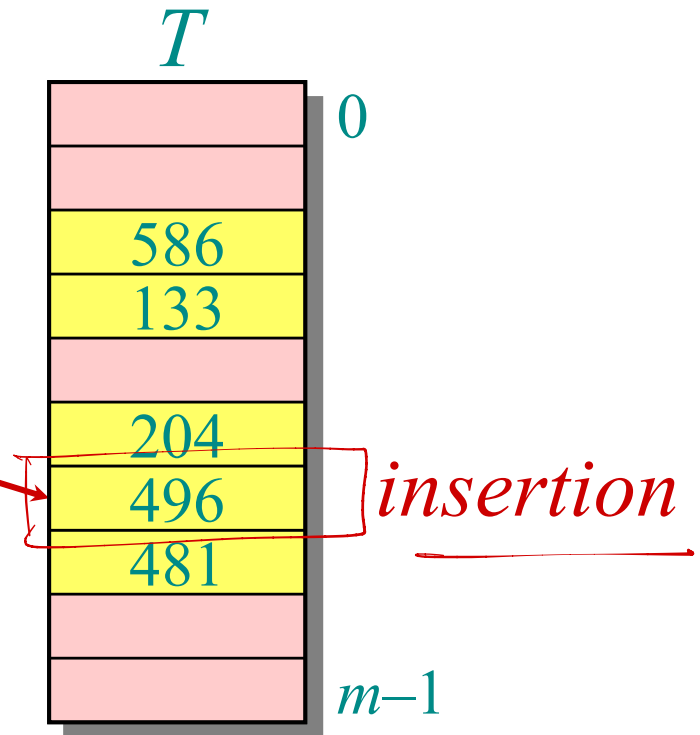
Insert key $k = 496$:

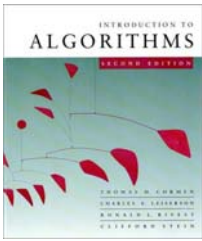
0. Probe $h(496, 0)$

1. Probe $h(496, 1)$

2. Probe $h(496, 2)$

3rd probe





Example of open addressing

Search for key $k = 496$:

0. Probe $h(496, 0)$
1. Probe $h(496, 1)$
2. Probe $h(496, 2)$

Search uses the same probe sequence, terminating successfully if it finds the key and unsuccessfully if it encounters an empty slot.

T

	586
	133
	204
	496
	481

Three cases:

0 ① You find the key in k^{th} probe

⇒ declare found!

② You find an empty slot in k^{th} probe

⇒ declare Not found!

③ Run out of probes & not find desired key ⇒ declare Not found!

Hash - Insert (T, k)

$i = 0$

repeat

$q = h(k, i)$

 if $T[q] == \text{NIL}$

$T[q] = k$

 return q

 else $i = i + 1$

until $i == m$

error "Hash table overflow"

Hash - Search (T, k)

$i = 0$

repeat

$q = h(k, i)$

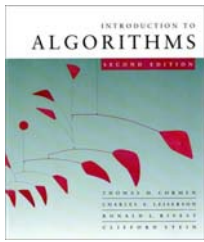
 if $T[q] == k$

 return q

$i = i + 1$

until $T[q] == \text{NIL}$ or $i == m$

return NIL



Probing strategies

Linear probing:

Given an ordinary hash function $h'(k)$, linear probing uses the hash function

$$h(k, i) = (h'(k) + i) \bmod m.$$

linear in i

$$\begin{aligned} i=0 &, h(k, 0) = h'(k) \bmod m \\ i=1 &, h(k, 1) = (h'(k) + 1) \bmod m \\ &\vdots \\ i=m-1 &, h(k, m-1) = (h'(k) + m-1) \bmod m \end{aligned}$$

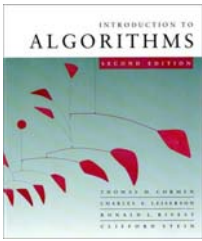
This method, though simple, suffers from **primary clustering**, where long runs of occupied slots build up, increasing the average search time. Moreover, the long runs of occupied slots tend to get longer.

Quadratic probing

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

quadratic in i

Suffixes form a milder form of clustering called
"Secondary clustering"



Probing strategies

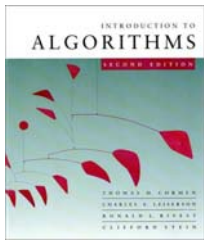
Double hashing

Given two ordinary hash functions $h_1(k)$ and $h_2(k)$, double hashing uses the hash function

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m.$$

This method generally produces excellent results, but $h_2(k)$ must be relatively prime to m . One way is to make m a power of 2 and design $h_2(k)$ to produce only odd numbers.

choose to be
coprimes

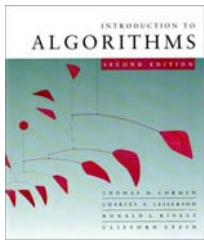


Analysis of open addressing

We make the assumption of *uniform hashing*:

- Each key is equally likely to have any one of the $m!$ permutations as its probe sequence.

Theorem. Given an open-addressed hash table with load factor $\alpha = n/m < 1$, the expected number of probes in an unsuccessful search is at most $1/(1-\alpha)$.



Proof of the theorem

Proof.

Unsuccessful search $\rightarrow$ find an empty slot
or
run out of slots to examine.

- At least one probe is always necessary.
- With probability n/m , the first probe hits an occupied slot, and a second probe is necessary.
- With probability $(n-1)/(m-1)$, the second probe hits an occupied slot, and a third probe is necessary.
- With probability $(n-2)/(m-2)$, the third probe hits an occupied slot, etc.

Observe that $\frac{n-i}{m-i} < \frac{n}{m} = \alpha$ for $i = 1, 2, \dots, n$.

Proof: $\frac{n-i}{m-i} < \frac{n}{m}$

; we know that $\frac{n}{m} = \alpha$

Let $\frac{n-i}{m-i} = \alpha'$

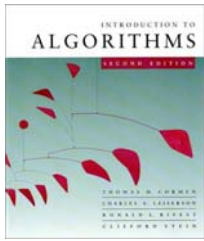
$\Rightarrow (n-i) = \alpha' (m-i)$

$\Rightarrow m\alpha - i = m\alpha' - i\alpha'$

$\Rightarrow \alpha = \alpha' + \frac{i}{m} (1 - \alpha')$

α' positive

$\Rightarrow \boxed{\alpha > \alpha'}$



Proof (continued)

in an
unsuccessful
search.

Therefore, the expected number of probes is

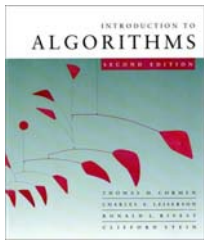
$$1 + \frac{n}{m} \left(1 + \frac{n-1}{m-1} \left(1 + \frac{n-2}{m-2} \left(\dots \left(1 + \frac{1}{m-n+1} \right) \dots \right) \right) \right)$$

$$\leq 1 + \alpha(1 + \alpha(1 + \alpha(\dots(1 + \alpha)\dots)))$$

$$\leq 1 + \alpha + \alpha^2 + \alpha^3 + \dots$$

$$= \sum_{i=0}^{\infty} \alpha^i$$
$$= \frac{1}{1 - \alpha} \cdot \square$$

The textbook has a more rigorous proof and an analysis of successful searches.



Implications of the theorem

- If α is constant, then accessing an open-addressed hash table takes constant time.
- If the table is half full, then the expected number of probes is $1/(1-0.5) = 2$.
- If the table is 90% full, then the expected number of probes is $1/(1-0.9) = 10$.

Successful search §

Theorem § Given an open addressed hash table with load factor $\alpha = \frac{n}{m} < 1$, the expected number of probes in a successful search is at most $\frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right)$ assuming indep. permutation hashing with no deletions & that each key is equally likely to be searched for.